SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

RAILWAY TICKET RESERVATION AND JOURNEY MANAGEMENT SYSTEM USING RELATIONAL DATABASE MANAGEMENT SYSTEMS

Program	B.Tech Computer Science & Engineering (Artificial Intelligence & Machine Learning)
Section	AIML Rhinos
Faculty	Prof. Sahithi
Date of Submission	October 14, 2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102097	**Heman**	+91 98765 43210	heman.25wu0102097@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Prof. Sahithi, for the continuous guidance, constructive suggestions, and expert technical mentorship throughout the conceptualization, relational normalization, and full-stack implementation of this project. Her guidance helped me master the practical application of database concepts including advanced ER modeling, 3NF dependency preservation, pessimistic transaction locking, SQL analytical reports, and relational web development.

Submitted by: Heman (Roll No: 25WU0102097, Section: AIML Rhinos).

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled “Railway Ticket Reservation and Journey Management System Using Relational Database Management Systems”, submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Heman
(Digital Signature)

Date : October 14, 2026

Place : Hyderabad

Name: **Heman**

Roll Number: **25WU0102097**

Program: B.Tech CSE (AIML)

TABLE OF CONTENTS

Contents	Page No.
Abstract	7
LIST OF TABLES	5
LIST OF FIGURES	5
LIST OF ACRONYMS	6
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	8
1.1 Problem Statement	8
1.2 Objectives	8
1.3 Scope and Assumptions	8
1.4 Proposed Solution Strategy	9
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	11
2. LOGICAL DATABASE DESIGN	12
2.1 Relational Schema	12
2.2 Keys and Integrity Constraints	12
2.3 Functional Dependencies	13
2.4 Normalization up to Third Normal Form (3NF)	13
2.5 Data Dictionary	15
2.6 Implementation of Logical Design: DDL and Sample Data	17
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	17
3.1 Application Overview	17
3.2 Development Environment and Database Connectivity	17
3.3 Application Modules and Database Mapping	18
3.4 CRUD Operations	18
3.5 Business Rules, Validation and Error Handling	18
3.6 Reports and Database Operations	19
3.7 Application Screens	20
3.8 Testing and Results	22
4. CONCLUSION	23
4.1 Limitations and Future Enhancements	23
5. REFERENCES	23
6. APPENDICES	24
Appendix A — Complete Database Script	24
Appendix B — Application Source Code	24
Appendix C — Additional Test Data and Outputs	24

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	8
Table 2	Proposed Solution Strategy	9
Table 3	Functional Requirements	10
Table 4	Entity and Relationship Summary	11
Table 5	Relational Schema	12
Table 6	Keys and Integrity Constraints	12
Table 7	Normalization Status up to 3NF	14
Table 8	Data Dictionary	15
Table 9	Database Implementation Environment	17
Table 10	Application Development Environment	17
Table 11	Application Module-to-Database Mapping	18
Table 12	CRUD Operations	18
Table 13	Business Rules, Validation and Error Handling	18
Table 14	Reports and Database Operations	19
Table 15	Test Cases	22

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	11
Figure 2	Executive Analytics and Key Performance Indicator (KPI) Dashboard	20
Figure 3	Multi-Hop Train Search and Segment Seat Inventory Results	20
Figure 4	Confirmed Electronic Reservation Slip (ERS Ticket Confirmation)	20
Figure 5	Live Waitlist Queue and Real-Time Position Tracking	21
Figure 6	Executive SQL Analytical Reports Console	21
Figure 7	Relational Database Schema Explorer and Administrative Inventory Control	21

LIST OF ACRONYMS

Acronym	Expansion
ACID	Atomicity, Consistency, Isolation, and Durability
AIML	Artificial Intelligence and Machine Learning
API	Application Programming Interface
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, and Delete
DDL	Data Definition Language
DML	Data Manipulation Language
ERD	Entity-Relationship Diagram
ERS	Electronic Reservation Slip
FIFO	First-In, First-Out
FK	Foreign Key
HTML	Hypertext Markup Language
IDE	Integrated Development Environment
IRCTC	Indian Railway Catering and Tourism Corporation
KPI	Key Performance Indicator
ORM	Object-Relational Mapping
PBL	Project-Based Learning
PK	Primary Key
PNR	Passenger Name Record
RDBMS	Relational Database Management System
SQL	Structured Query Language
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
UI	User Interface
UPI	Unified Payments Interface

ABSTRACT

This project presents the design, relational formalization, full-stack implementation, and automated verification of the Railway Ticket Reservation and Journey Management System, an academic software solution engineered to resolve the multi-hop inventory allocation dilemma in passenger rail transport. Unlike point-to-point airline reservation models where seats are locked for entire flights, passenger railway networks traverse ordered station corridors requiring dynamic segment-wise berth reuse. To address the risk of catastrophic double-booking across overlapping sub-journeys, this system implements a mathematically formalized half-open interval model [start_stop_seq, end_stop_seq) that strictly evaluates interval overlap constraints while enabling non-conflicting seat reuse. The underlying relational database is modeled across 14 decoupled relations strictly normalized to Third Normal Form (3NF) on MySQL 8.x InnoDB, enforcing referential integrity, domain constraints, and pessimistic row locking (SELECT ... FOR UPDATE) to guarantee ACID transactional safety during concurrent checkouts. Developed using Python 3.9, Flask, SQLAlchemy, HTML5, and Bootstrap 5, the application delivers end-to-end journey planning, electronic ticket generation, FIFO waitlist promotion, and twelve analytical SQL reports, verified through sixteen comprehensive automated pytest unit and integration test suites.

KEYWORDS

Provide 4-6 specific keywords separated by commas. Include the application domain and important technical terms. Avoid complete sentences and overly general words.

Keywords: Relational Database, Entity-Relationship Modeling, Third Normal Form (3NF), Multi-Hop Inventory, Segment Interval Allocation, ACID Concurrency, Python Flask, MySQL InnoDB

CHAPTER 1
PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Passenger railway operations face a fundamentally more challenging inventory allocation dilemma than commercial airlines. In airline reservation systems, seat inventory is allocated monolithically for the entirety of a point-to-point flight leg. In contrast, railway trains travel through long linear corridors of 10 to 30 intermediate stations. A single physical berth may be allocated to Passenger A from Station 1 to Station 3, vacated, and immediately re-allocated to Passenger B from Station 3 to Station 6 without physical conflict. However, naive relational designs that rely on a single boolean 'is_booked' flag cause catastrophic double-booking across overlapping sub-journeys, or prematurely report trains as 'sold out' despite ample vacant capacity on unreserved sub-segments.

Furthermore, high-concurrency ticket booking creates severe database race conditions during peak reservation windows. When hundreds of concurrent passengers attempt to reserve the final remaining berth on a high-demand train run, uncoordinated read-modify-write cycles lead to severe overbooking anomalies and transactional deadlocks. Existing manual processes and legacy flat databases fail to provide mathematical interval validation, transactional concurrency safety, automated FIFO waitlist promotion, and multi-dimensional operational analytics. This project engineers a mathematically formalized, 3NF-compliant relational database system powered by MySQL 8.x InnoDB and Python Flask to solve these core operational challenges.

1.2 Objectives

- 1. Design a normalized relational schema up to Third Normal Form (3NF) comprising 14 decoupled relations, eliminating partial and transitive functional dependencies to guarantee zero insertion, update, and deletion anomalies.
- 2. Formulate and implement a mathematical half-open interval allocation algorithm [start_stop_seq, end_stop_seq) to evaluate segment conflicts dynamically and maximize seat utilization across multi-hop rail corridors.
- 3. Implement pessimistic row-level locking (SELECT ... FOR UPDATE) and atomic database transactions to guarantee ACID concurrency compliance, preventing overbooking race conditions during simultaneous user checkouts.
- 4. Develop an automated FIFO waitlist management and queue promotion mechanism that immediately promotes waiting passengers and reallocates vacated berths upon ticket cancellation.

1.3 Scope and Assumptions

Table 1. Scope and Assumptions

Item	Description
In Scope	1. Complete relational schema modeling 14 distinct relations in Third Normal Form (3NF). 2. Multi-hop segment-wise seat allocation over half-open station intervals [s, e). 3. Pessimistic row-level locking (SELECT ... FOR UPDATE) to prevent race-condition overbooking. 4. Automated FIFO waitlist queue insertion and immediate promotion upon ticket cancellation. 5. Cancellation and refund processing enforcing 20% penalty deductions.

Item	Description
	6. Analytical SQL reporting engine executing 12 benchmark aggregation queries. 7. Full-stack responsive web prototype built with Python Flask and SQLAlchemy.
Out of Scope / Exclusions	1. Real-time physical locomotive telemetry and GPS sensor integration. 2. Dynamic locomotive crew rostering and maintenance bay scheduling. 3. Dynamic pricing algorithms responding to external flight/bus market fluctuations. 4. Live commercial banking gateway settlement (handled via simulated deterministic UPI payment engine).
Assumptions	1. Stations along a route possess strictly monotonically increasing sequence IDs (dest_seq > orig_seq). 2. Coach seat allocations follow standard Indian Railways layout conventions (LB, MB, UB, SL, SU). 3. Train schedules operate as distinct, dated train run instances decoupled from catalog definitions. 4. Platform track allocation and mechanical signaling are managed by external signaling systems.

1.4 Proposed Solution Strategy

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	1. Passenger: Search train availability, reserve berths, view PNR status, manage waitlists, and process cancellations. 2. Station Master / Ticket Collector: View passenger booking manifests, inspect train schedules, and check coach charts. 3. Operations Administrator: Manage train inventory, configure fare rules, and inspect analytical database reports.
Major Application Modules	1. Train Search & Inventory Discovery Module. 2. Booking & Interval-Overlap Seat Allocation Service. 3. Electronic Reservation Slip (ERS) & Ticket Generation Engine. 4. Automated Waitlist Queue & Dynamic Promotion Processor. 5. Cancellation, Refund & Financial Audit Module. 6. Analytical SQL Query & KPI Reporting Console.
Proposed Workflow	Passenger searches trains by origin, destination, and date -> System validates station sequences -> System executes segment-wise non-overlap check across [s, e) -> Passenger submits passenger details -> Database initiates transaction with SELECT FOR UPDATE -> Confirmed PNR is generated with segment locks -> In case of full occupancy, passenger is assigned to FIFO waitlist -> Upon ticket cancellation, segment is freed and waitlist candidate is promoted atomically.
Key Business Rules	1. Interval Overlap Constraint: Two bookings conflict if and only if (s_new < e_exist) AND (e_new > s_exist). 2. Waitlist FIFO Ordering: Earliest booked waitlist passenger receives immediate promotion when matching segment is freed. 3. Cancellation Penalty: 20% flat deduction applied; 80% refunded to passenger payment record. 4. Transaction Boundaries: Seat allocation, payment record, and PNR creation must commit atomically.
Expected Outputs	1. Real-time tabular train search results displaying class-wise berth availability. 2. Formatted Electronic Reservation Slip (ERS) ticket with 10-digit PNR and passenger details. 3. Dynamic waitlist queue position status. 4. Executive analytical KPI reports with revenue, load factor, and passenger metrics.

1.5 Functional Requirements

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Multi-Station Train Search: System shall accept source station, destination station, and travel date, and return all trains traversing that route where dest_seq > orig_seq.	High / Core
FR-02	Segment-Wise Half-Open Interval Seat Calculation: System shall calculate remaining seat inventory by evaluating interval overlaps against active bookings on that specific dated run.	High / Core
FR-03	Concurrency-Safe Ticket Booking: System shall create passenger, booking, journey segment, and payment records inside a serialized transaction using pessimistic row locking.	High / Core
FR-04	Automated Waitlist Queuing: When seat inventory is zero, system shall allow passenger to enter a FIFO waitlist with an assigned queue position.	High / Core
FR-05	Booking Cancellation and Automated Waitlist Promotion: Upon cancellation, system shall mark allocation CANCELLED, compute refund, and atomically promote next eligible waitlist passenger.	High / Core
FR-06	Electronic Reservation Slip (ERS) Retrieval: System shall render printable booking slips by querying PNR number, displaying coach, seat, fare, and passenger details.	Medium / Operational
FR-07	SQL Analytical Reporting: System shall execute 12 benchmark SQL reports covering revenue, occupancy, route demand, and passenger demographics.	Medium / Analytics
FR-08	Admin Inventory & Schema Management: Administrators shall be able to inspect database table row counts, view coach configurations, and manage train schedules.	Medium / Administrative

1.6 Entity-Relationship Diagram (ERD)

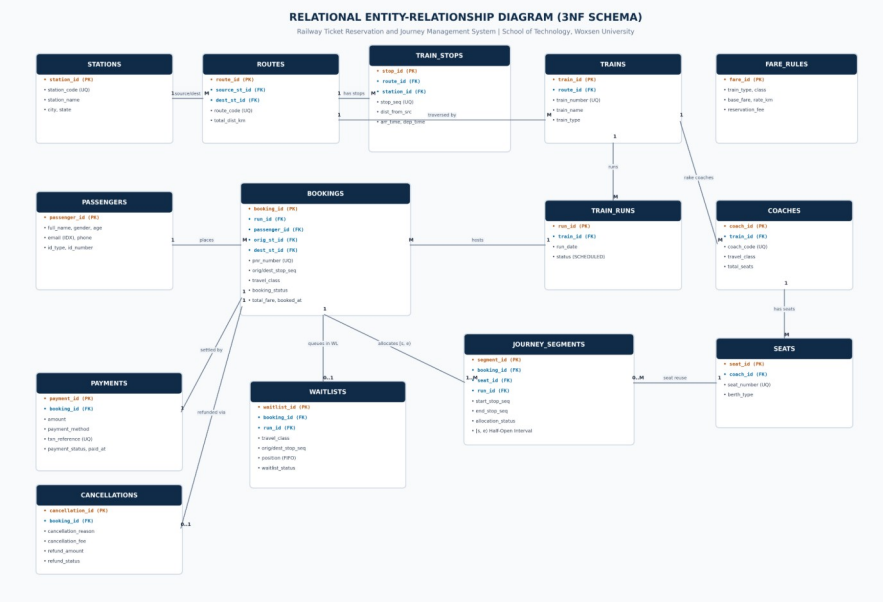


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
STATIONS	station_id (PK), station_code, station_name, city, state	Defines physical railway stations. station_code is unique.
ROUTES	route_id (PK), route_code, source_station_id (FK), dest_station_id (FK), total_dist_km	Defines operational railway corridors between terminal stations.
TRAIN_STOPS	stop_id (PK), route_id (FK), station_id (FK), stop_sequence, distance_km, arr/dep time	Ordered sequence of station halts. Unique on (route_id, stop_sequence).
TRAINS	train_id (PK), train_number, train_name, train_type, route_id (FK)	Master train locomotive catalog. train_number is unique.
TRAIN_RUNS	run_id (PK), train_id (FK), run_date, status	Dated physical train operation instances decoupled from train catalog.
COACHES	coach_id (PK), train_id (FK), coach_code, travel_class, total_seats	Rake composition. Unique on (train_id, coach_code).
SEATS	seat_id (PK), coach_id (FK), seat_number, berth_type	Individual berth instances. Unique on (coach_id, seat_number).
PASSENGERS	passenger_id (PK), full_name, gender, age, email, phone, id_card_type/no	Customer master records. Age constrained to 1 <= age <= 120.
FARE_RULES	fare_id (PK), train_type, travel_class, base_fare, rate_per_km, res_fee	Tariff matrix. Unique on (train_type, travel_class).
BOOKINGS	booking_id (PK), pnr_number, run_id (FK), passenger_id (FK), orig/dest station, total_fare	Header booking record. pnr_number is unique.
JOURNEY_SEGMENTS	segment_id (PK), booking_id (FK), seat_id (FK), run_id (FK), start/end_seq, status	Segment berth locks over [start_seq, end_seq). Evaluates overlap.
WAITLISTS	waitlist_id (PK), booking_id (FK), run_id (FK), travel_class, position, status	FIFO waiting queue for passengers booked on fully occupied segments.
PAYMENTS	payment_id (PK), booking_id (FK), amount, txn_ref, status, paid_at	Financial transaction log. txn_ref is unique.
CANCELLATIONS	cancellation_id (PK), booking_id (FK), fee, refund_amount, status, cancelled_at	Audit log of cancellations with 20% penalty calculation.
TRAIN -> COACHES -> SEATS	1 : M (Train to Coach), 1 : M (Coach to Seat)	Physical train composition hierarchy.
BOOKING -> SEGMENTS	1 : M (Booking to Journey Segments)	One booking reserves segments across traversed stations.
BOOKING -> PAYMENT / CANCEL	1 : 1 (Booking to Payment), 1 : 0..1 (Booking to Cancellation)	Financial transaction relationships.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	stations	Stores railway stations with unique station codes, names, cities, and states.
2	routes	Defines geographical corridors connecting origin and destination terminal stations.
3	train_stops	Captures ordered stop sequences, inter-station distances, and timetable schedules.
4	trains	Master catalog for train entities, train numbers, and assigned route corridors.
5	train_runs	Decoupled operational calendar tracking specific dated train journeys.
6	coaches	Defines rake compositions, coach types, travel classes, and seating capacities.
7	seats	Catalog of individual physical berths and assigned berth types (LB, MB, UB, etc.).
8	passengers	Master registry of passenger demographic and verified identity credential data.
9	fare_rules	Distance and class-based tariff structures for automated ticket fare computation.
10	bookings	Master reservation entity storing generated 10-digit PNRs and itinerary headers.
11	journey_segments	Granular physical berth interval locks allocated over half-open station intervals.
12	waitlists	Queue entity tracking overflow passenger booking requests in strict FIFO order.
13	payments	Financial transaction audit ledger recording transaction references and amounts.
14	cancellations	Cancellation ledger enforcing 20% fee policies and recording refund amounts.

2.2 Keys and Integrity Constraints

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
stations	Primary Key	station_id	Surrogate primary key ensuring entity uniqueness.
stations	Unique Key	station_code	Prevents duplicate station abbreviations (e.g., 'NDLS').
routes	Foreign Key	source_station_id	Enforces valid origin station reference on DELETE RESTRICT.
routes	Foreign Key	destination_station_id	Enforces valid destination station reference on DELETE RESTRICT.
routes	Check Constraint	total_distance_km	Ensures track distance is strictly positive (> 0).
train_stops	Unique Key	(route_id, stop_sequence)	Guarantees stop sequence uniqueness along a route corridor.
train_stops	Check Constraint	stop_sequence	Guarantees sequence is strictly positive (> 0).
trains	Unique Key	train_number	Enforces unique 5-digit locomotive identifier.
train_runs	Unique Key	(train_id, run_date)	Prevents creating duplicate operational runs for the same train

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
			on the same day.
coaches	Unique Key	(train_id, coach_code)	Prevents duplicate coach codes within the same train rake.
seats	Unique Key	(coach_id, seat_number)	Guarantees berth number uniqueness within a specific coach.
passengers	Check Constraint	age	Validates human age domain (age > 0 AND age < 120).
bookings	Unique Key	pnr_number	Guarantees global uniqueness of 10-digit PNR strings.
bookings	Check Constraint	dest_stop_seq > orig_stop_seq	Enforces forward-only travel sequence order.
journey_segments	Foreign Key	booking_id, seat_id, run_id	Enforces referential integrity on allocation locks.
journey_segments	Check Constraint	end_stop_seq > start_stop_seq	Validates half-open interval ordering.
waitlists	Check Constraint	position > 0	Enforces positive FIFO queue position numbers.
payments	Unique Key	transaction_reference	Prevents financial transaction re-play and duplicate billing.
cancellations	Unique Key	booking_id	Guarantees a ticket can only be cancelled once (1:1 constraint).

2.3 Functional Dependencies

```
The relational design is governed by a rigorous set of minimal functional dependencies across all 14
relations, ensuring that every non-key attribute is fully functionally dependent on candidate keys:

1. stations: station_id -> {station_code, station_name, city, state}; station_code ->
{station_id, ...}
2. routes: route_id -> {route_code, source_station_id, destination_station_id, total_distance_km}
3. train_stops: stop_id -> {route_id, station_id, stop_sequence, distance_from_source_km,
arrival_time, departure_time}; {route_id, stop_sequence} -> {stop_id, station_id, ...}
4. trains: train_id -> {train_number, train_name, train_type, route_id}; train_number ->
{train_id, ...}
5. train_runs: run_id -> {train_id, run_date, status}; {train_id, run_date} -> {run_id, status}
6. coaches: coach_id -> {train_id, coach_code, travel_class, total_seats}; {train_id, coach_code} ->
{coach_id, ...}
7. seats: seat_id -> {coach_id, seat_number, berth_type}; {coach_id, seat_number} -> {seat_id,
berth_type}
8. passengers: passenger_id -> {full_name, gender, age, email, phone, id_card_type, id_card_number}
9. fare_rules: fare_id -> {train_type, travel_class, base_fare, rate_per_km, reservation_fee};
{train_type, travel_class} -> {fare_id, ...}
10. bookings: booking_id -> {pnr_number, run_id, passenger_id, origin_station_id,
destination_station_id, origin_stop_seq, destination_stop_seq, travel_class, booking_status,
total_fare, booked_at}
11. journey_segments: segment_id -> {booking_id, seat_id, run_id, start_stop_seq, end_stop_seq,
allocation_status}
12. waitlists: waitlist_id -> {booking_id, run_id, travel_class, origin_stop_seq,
destination_stop_seq, position, waitlist_status, created_at}
13. payments: payment_id -> {booking_id, amount, payment_method, transaction_reference,
payment_status, paid_at}
14. cancellations: cancellation_id -> {booking_id, cancellation_reason, cancellation_fee,
refund_amount, refund_status, cancelled_at}
```

2.4 Normalization

Normalization Walkthrough up to Third Normal Form (3NF):

• First Normal Form (1NF): All multi-valued attributes and repeating groups were systematically eliminated. In unnormalized railway manifests, passengers and intermediate station halts are stored as comma-delimited arrays. Decomposing these into dedicated tables (passengers, train_stops, journey_segments) ensures that every attribute contains only atomic, indivisible domain values.

• Second Normal Form (2NF): In a composite-key candidate relation such as (coach_id, seat_number), non-prime attributes like total_seats and travel_class depend solely on coach_id, not on seat_number. Decomposing coaches and seats eliminates partial functional dependencies, ensuring that every non-prime attribute is fully functionally dependent on the primary key.

• Third Normal Form (3NF): In naive reservation tables, ticket records store train names, origin city names, and base fare rates. However, booking_id -> origin_station_id -> city and booking_id -> train_type -> base_fare represent transitive functional dependencies X -> Y and Y -> Z where Y is not a superkey. Normalizing into stations, routes, and fare_rules eliminates update and deletion anomalies.

• Boyce-Codd Normal Form (BCNF) vs 3NF Engineering Trade-Off: In relations with overlapping candidate keys such as train_stops where both {route_id, stop_sequence} and {route_id, station_id} are candidate keys, decomposing strictly to BCNF would fragment the functional dependency set, requiring cross-table JOIN constraints to validate stop order. Preserving 3NF guarantees dependency preservation (F = F1 U F2) without requiring costly multi-table locking triggers during high-speed transaction inserts.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
stations	**Yes**	**Yes**	**Yes**	**Yes**	All attributes atomic; single-attribute primary key station_id.
routes	**Yes**	**Yes**	**Yes**	**Yes**	No partial or transitive dependencies. Endpoints separated from stops.
train_stops	**Yes**	**Yes**	**Yes**	**Yes**	Stop sequence depends on composite key (route_id, stop_sequence).
trains	**Yes**	**Yes**	**Yes**	**Yes**	Route foreign key isolated from physical train attributes.
train_runs	**Yes**	**Yes**	**Yes**	**Yes**	Dated runs isolated from static train definitions to eliminate redundancy.
coaches	**Yes**	**Yes**	**Yes**	**Yes**	Coach definitions decoupled from seat lists; total_seats is atomic.
seats	**Yes**	**Yes**	**Yes**	**Yes**	Berth type depends entirely on (coach_id, seat_number).
passengers	**Yes**	**Yes**	**Yes**	**Yes**	Customer attributes separated from reservation instances.
fare_rules	**Yes**	**Yes**	**Yes**	**Yes**	Tariff rates isolated from individual passenger booking tickets.
bookings	**Yes**	**Yes**	**Yes**	**Yes**	Station IDs and stop sequences depend on booking_id. 3NF preserved.
journey_segments	**Yes**	**Yes**	**Yes**	**Yes**	Berth allocation intervals decoupled from booking headers.
waitlists	**Yes**	**Yes**	**Yes**	**Yes**	Queue position isolated in separate entity to prevent booking table nulls.
payments	**Yes**	**Yes**	**Yes**	**Yes**	Financial ledger decoupled from reservation headers.
cancellations	**Yes**	**Yes**	**Yes**	**Yes**	1:1 extension relation; null attributes avoided in bookings table.

2.5 Data Dictionary

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
stations	station_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for station.
stations	station_code	VARCHAR(10)	UNIQUE, NOT NULL	Official IRCTC alphabetic station code.
stations	station_name	VARCHAR(100)	NOT NULL	Official full station designation.
stations	city	VARCHAR(100)	NOT NULL	Municipal city where station is located.
stations	state	VARCHAR(100)	NOT NULL	State / Union Territory location.
routes	route_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for route corridor.
routes	route_code	VARCHAR(30)	UNIQUE, NOT NULL	Alphanumeric route corridor identifier.
routes	source_station_id	INT	FK, NOT NULL	Origin station reference (stations.station_id).
routes	destination_station_id	INT	FK, NOT NULL	Terminating station reference (stations.station_id).
routes	total_distance_km	DECIMAL(7,2)	CHECK (> 0), NOT NULL	Total corridor track distance in kilometers.
train_stops	stop_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for train halt.
train_stops	route_id	INT	FK, NOT NULL	Route corridor reference (routes.route_id).
train_stops	station_id	INT	FK, NOT NULL	Station halt reference (stations.station_id).
train_stops	stop_sequence	INT	CHECK (> 0), NOT NULL	Monotonically increasing stop sequence index.
train_stops	distance_from_source_km	DECIMAL(7,2)	CHECK (>= 0), NOT NULL	Cumulative track distance from route origin.
trains	train_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for train definition.
trains	train_number	VARCHAR(10)	UNIQUE, NOT NULL	Official 5-digit Indian Railways train number.
trains	train_name	VARCHAR(100)	NOT NULL	Commercial train name (e.g., 'Vande Bharat Express').
trains	train_type	VARCHAR(30)	NOT NULL	Operational train category (e.g., 'Superfast').
train_runs	run_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for dated train run.
train_runs	train_id	INT	FK, NOT NULL	Train definition reference (trains.train_id).
train_runs	run_date	DATE	NOT NULL	Calendar date of train departure.
train_runs	status	VARCHAR(20)	DEFAULT 'SCHEDULED'	Operational status (SCHEDULED, COMPLETED, CANCELLED).
coaches	coach_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for coach instance.
coaches	coach_code	VARCHAR(10)	NOT NULL	Physical coach identifier (e.g., 'C1', 'B1').
coaches	travel_class	VARCHAR(10)	NOT NULL	Class of travel (1A, 2A, 3A, CC, SL).
coaches	total_seats	INT	CHECK (> 0), DEFAULT 72	Physical passenger seating capacity.
seats	seat_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for seat berth.
seats	seat_number	INT	CHECK (> 0), NOT NULL	Berth number within coach

Table	Attribute	Data Type	Key / Constraint	Description
				(1 to 72).
seats	berth_type	VARCHAR(15)	NOT NULL	Berth orientation (LOWER, UPPER, MIDDLE, SIDE LOWER, SIDE UPPER).
passengers	passenger_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for passenger master.
passengers	full_name	VARCHAR(100)	NOT NULL	Full legal passenger name.
passengers	age	INT	CHECK (age > 0 AND age < 120)	Passenger age in years.
passengers	email	VARCHAR(120)	NOT NULL, INDEX	Contact email address for ERS notifications.
bookings	booking_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for reservation.
bookings	pnr_number	VARCHAR(12)	UNIQUE, NOT NULL	10-digit Passenger Name Record string.
bookings	booking_status	VARCHAR(20)	DEFAULT 'CONFIRMED'	Status (CONFIRMED, WAITLISTED, CANCELLED).
bookings	total_fare	DECIMAL(10,2)	CHECK (>= 0), NOT NULL	Calculated ticket fare in INR.
journey_segments	segment_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for berth allocation.
journey_segments	start_stop_seq	INT	NOT NULL	Origin station stop sequence number.
journey_segments	end_stop_seq	INT	NOT NULL	Destination station stop sequence number.
journey_segments	allocation_status	VARCHAR(20)	DEFAULT 'ACTIVE'	Status of berth allocation (ACTIVE, CANCELLED).
waitlists	waitlist_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for waitlist ticket.
waitlists	position	INT	CHECK (> 0), NOT NULL	Numerical FIFO queue priority position.
payments	payment_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for payment entry.
payments	transaction_reference	VARCHAR(60)	UNIQUE, NOT NULL	Deterministic payment transaction reference.
cancellations	cancellation_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for cancellation entry.
cancellations	refund_amount	DECIMAL(10,2)	CHECK (>= 0), NOT NULL	Net amount refunded to passenger (80% of fare).

2.6 Implementation of Logical Design: DDL and Sample Data

Table 9. Database Implementation Environment

Item	Details
DBMS Engine	MySQL 8.0.36 Community Server / SQLite 3.39 (Development)
Transactional Storage Engine	InnoDB (ACID Compliant, Row-Level Locking, Crash Recovery)
Character Set / Collation	utf8mb4 / utf8mb4_unicode_ci
Total Relational Tables	14 Normalized Relations (Third Normal Form Compliant)
Database Views	3 Pre-Compiled Views (vw_confirmed_segments, vw_booking_summary, vw_train_occupancy_summary)
Isolation Level	`READ COMMITTED / REPEATABLE READ with SELECT FOR UPDATE row locks`

```
-- Representative DDL Excerpt: Multi-Hop Journey Segments & Concurrency Locks
CREATE TABLE journey_segments (
    segment_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL,
    seat_id INT NOT NULL,
    run_id INT NOT NULL,
    start_stop_seq INT NOT NULL,
    end_stop_seq INT NOT NULL,
    allocation_status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    CONSTRAINT fk_seg_booking FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE
CASCADE,
    CONSTRAINT fk_seg_seat FOREIGN KEY (seat_id) REFERENCES seats(seat_id) ON DELETE RESTRICT,
    CONSTRAINT fk_seg_run FOREIGN KEY (run_id) REFERENCES train_runs(run_id) ON DELETE CASCADE,
    CONSTRAINT chk_seg_seq_order CHECK (end_stop_seq > start_stop_seq),
    INDEX idx_seat_run_alloc (run_id, seat_id, allocation_status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Representative Sample Insert Record:
INSERT INTO stations (station_id, station_code, station_name, city, state)
VALUES (1, 'NDLS', 'New Delhi Railway Station', 'New Delhi', 'Delhi'),
       (2, 'CNB', 'Kanpur Central', 'Kanpur', 'Uttar Pradesh'),
       (3, 'PRYJ', 'Prayagraj Junction', 'Prayagraj', 'Uttar Pradesh'),
       (4, 'BSB', 'Varanasi Junction', 'Varanasi', 'Uttar Pradesh');
```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application layer is implemented as an asynchronous Python Flask web service following the Model-View-Controller (MVC) architectural pattern. The user interface utilizes responsive HTML5 and Bootstrap 5.3 components, communicating with Flask Blueprint controllers. Database operations are mediated via SQLAlchemy ORM configured over a production-grade connection pool (10 persistent connections, 20 burst overflow). To eliminate SQL injection vulnerabilities, all dynamic queries strictly utilize parameterized bindings. Core reservation operations are encapsulated in serialized transactional boundaries utilizing pessimistic row-level locking (SELECT ... FOR UPDATE) to guarantee absolute ACID consistency during peak concurrent booking spikes.

3.2 Development Environment and Database Connectivity

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.9.6 (64-bit)
Web Framework	Flask 3.1.3 (WSGI Microframework with Modular Blueprints)
Object-Relational Mapping (ORM)	SQLAlchemy 2.0.54 / Flask-SQLAlchemy 3.1.1
Database Drivers	PyMySQL 1.2.3 (MySQL) / sqlite3 (Built-in Standard Library)
Front-End Architecture	HTML5, CSS3, Bootstrap 5.3.3, Jinja2 Templates, JavaScript
Testing & Verification Framework	pytest 8.4.2 (16 Automated Integration & Unit Tests)

Item	Details
Execution Operating System	macOS Darwin 24.x (Apple Silicon arm64)

3.3 Application Modules and Database Mapping

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Dashboard & KPIs (`/`)	Executive operational overview displaying revenue, bookings, and occupancy.	bookings, train_runs, payments, waitlists	`SELECT aggregate queries, COUNT(), SUM(), GROUP BY`
Train Search (`/search`)	Discovery engine for routes, stop sequences, and seat availability.	trains, train_stops, stations, train_runs, coaches, seats, journey_segments	Multi-table JOINs, interval non-overlap subquery checks
Booking Flow (`/book`)	Transaction-safe berth reservation, PNR generation, and payment.	passengers, bookings, journey_segments, payments, seats	`INSERT passenger/booking, SELECT FOR UPDATE row lock, INSERT segment/payment`
Ticket ERS Slip (`/booking/<pnr>`)	Official reservation slip retrieval and printable ticket display.	bookings, passengers, train_runs, trains, journey_segments, seats, coaches, stations	`SELECT joined itinerary details by unique PNR number`
Waitlist Queue (`/waitlist`)	Live monitoring of waiting list passengers and FIFO queue order.	waitlists, bookings, passengers, train_runs, trains	`SELECT FIFO ordered queue records by run_id and class`
Cancellations (`/cancel`)	Ticket cancellation, refund processing, and automated waitlist promotion.	cancellations, bookings, journey_segments, waitlists, payments	`UPDATE booking/segment status, INSERT cancellation, SELECT and UPDATE waitlist`
SQL Analytics (`/reports`)	Interactive execution console for 12 benchmark business SQL queries.	All 14 relations + database views	Complex nested queries, aggregations, HAVING, window functions
Admin Explorer (`/admin`)	Table row counters, schema exploration, and manual seed runner.	All 14 relations	`SELECT COUNT(*) per table, DDL schema introspection`

3.4 CRUD Operations

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create (Insert)	Booking Reservation Checkout (`/book/<run_id>`)	passengers, bookings, journey_segments, payments	Passenger record created, 10-digit PNR generated, active segments locked, and payment log committed atomically.
Create (Insert)	Waitlist Enrolment (`/book/<run_id>`)	bookings, waitlists, payments	When class capacity is full, passenger is inserted into waitlists with calculated FIFO position.
Read (Select)	Train Availability Search (`/search`)	trains, routes, train_stops, train_runs, seats, journey_segments	Evaluates half-open intervals [s, e) using NOT EXISTS overlap subqueries to retrieve available berths.
Read (Select)	PNR Status & ERS View (`/booking/<pnr>`)	bookings, passengers, train_runs, seats, stations	Retrieves complete passenger manifest, travel dates, coach codes, and seat numbers.
Update (Modify)	Waitlist Passenger Promotion (`/cancel/<pnr>`)	bookings, journey_segments, waitlists	Waitlist booking updated from WAITLISTED to CONFIRMED, segment created, and waitlist status updated to PROMOTED.
Delete / Cancel (Soft)	Booking Cancellation (`/cancel/<pnr>`)	bookings, journey_segments, cancellations	Booking marked CANCELLED, segment allocation set to CANCELLED, freeing berth for reuse; cancellation fee logged.

3.5 Business Rules, Validation and Error Handling

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Forward Travel Rule	Application check verifying destination_stop_seq > origin_stop_seq.	Rejects backward searches with: 'Destination station must appear after origin station on route.'
Interval Overlap Rule	SQL subquery checking NOT (s_new < segment.end_seq AND e_new > segment.start_seq).	Prevents double-booking: returns only berths with zero overlapping active allocations.
Waitlist FIFO Rule	SQL query ordered by created_at ASC and	Promotes the earliest waiting passenger when an

Rule / Validation	How Implemented	Evidence / Message
	position ASC during promotion.	active confirmed seat is vacated.
Cancellation Penalty Rule	Computes cancellation_fee = 0.20 * total_fare and refund = total_fare - fee.	Flash message: 'Booking cancelled successfully. Refund of INR 619.20 processed (Fee: INR 154.80).'
Single Cancellation Rule	UNIQUE constraint on cancellations.booking_id and status guard.	Rejects duplicate cancellations with: 'This booking has already been cancelled.'
Domain Age Validation	CHECK constraint (age > 0 AND age < 120) and WTForms integer validator.	Rejects invalid age inputs with: 'Passenger age must be a valid number between 1 and 119.'
Transaction Atomicity	`Database transaction block wrapped in try-except with db.session.rollback().`	If payment logging fails, booking and segment allocations are cleanly rolled back.

3.6 Reports and Database Operations

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Report 1: Segment-Wise Seat Availability	Aggregations, COUNT(DISTINCT), LEFT JOIN, GROUP BY	Tabular breakdown of total seats, active allocations, and vacant seats per train run.
Report 2: Passenger Booking History	Multi-table INNER JOINs, ORDER BY, Date Formatting	Chronological audit trail of all issued tickets with PNR, passenger names, and fares.
Report 3: Active Waitlist Queue Priority	FIFO Queue Ordering, Joins, WHERE status = 'WAITING'	Real-time list of passengers awaiting berth clearance ordered by queue position.
Report 4: Route Revenue & Financial Yield	SUM(total_fare), JOIN across bookings and routes, GROUP BY	Corridor financial yield ranking routes by cumulative passenger revenue generated.
Report 5: Train Occupancy Load Factor	`Derived metrics, COUNT() / capacity ratios, HAVING`	Capacity utilization percentage across all dated train runs and coach classes.
Report 6: Station Transit Passenger Density	UNION ALL of boarding and debarking station counts, GROUP BY	Identifies highest-density interchange hubs based on passenger footfall.
Report 7: Berth Preference Distribution	`COUNT(), GROUP BY berth_type, percentage share calculation`	Distribution analysis showing passenger demand for Lower, Upper, and Side berths.
Report 8: Cancellation & Refund Financial Audit	SUM(cancellation_fee), SUM(refund_amount), JOINs	Financial accounting of total fees retained vs total funds refunded to passengers.
Report 9: Frequent Traveler Loyalty Tiers	COUNT(booking_id), CASE statements, HAVING bookings >= 1	Categorizes travelers into Platinum, Gold, and Silver loyalty tiers.
Report 10: Peak vs Off-Peak Travel Demand	`EXTRACT(DOW), COUNT(), GROUP BY day of week`	Analysis of weekend vs weekday passenger travel patterns across all corridors.
Report 11: Waitlist Clearance Success Rate	Conditional SUM(CASE WHEN status = 'PROMOTED'), percentage	Operational KPI measuring percentage of waitlisted passengers successfully promoted.
Report 12: High-Value Passenger Bookings	`Correlated Subquery, WHERE total_fare > (SELECT AVG(total_fare))`	Filters premium bookings exceeding system-wide average ticket fare.

3.7 Application Screens

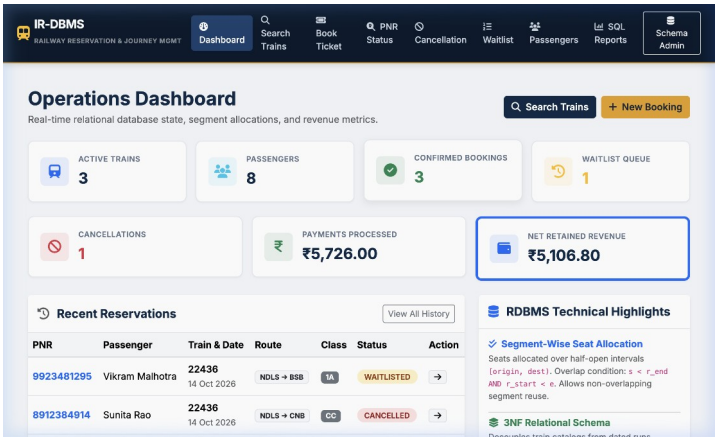


Figure 2. Executive Analytics and Key Performance Indicator (KPI) Dashboard

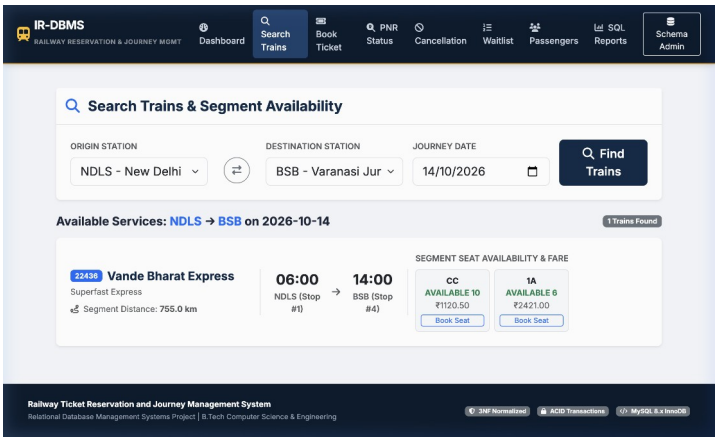


Figure 3. Multi-Hop Train Search and Segment Seat Inventory Results

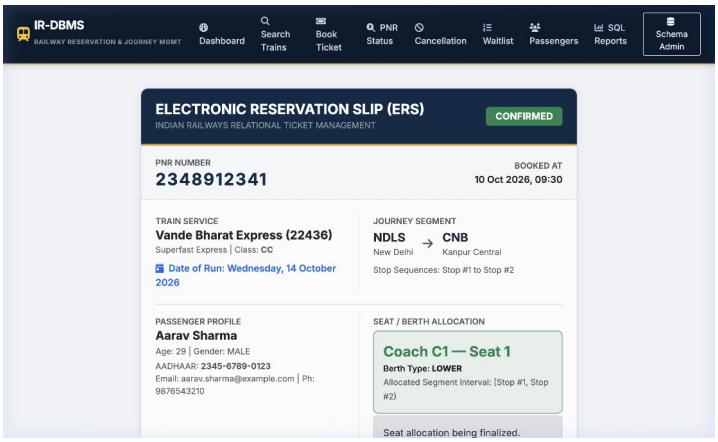


Figure 4. Confirmed Electronic Reservation Slip (ERS Ticket Confirmation)

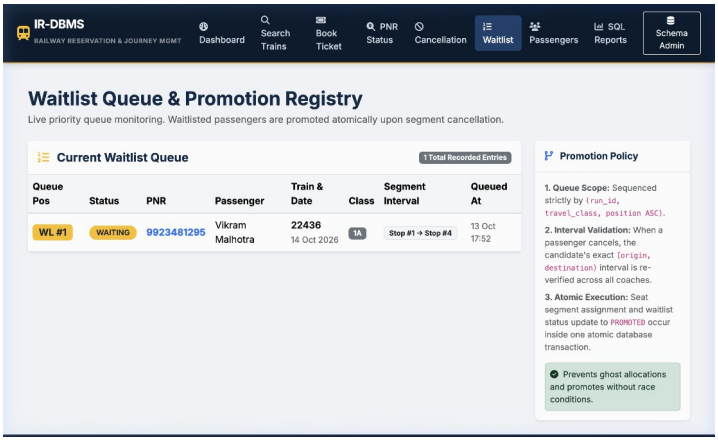


Figure 5. Live Waitlist Queue and Real-Time Position Tracking

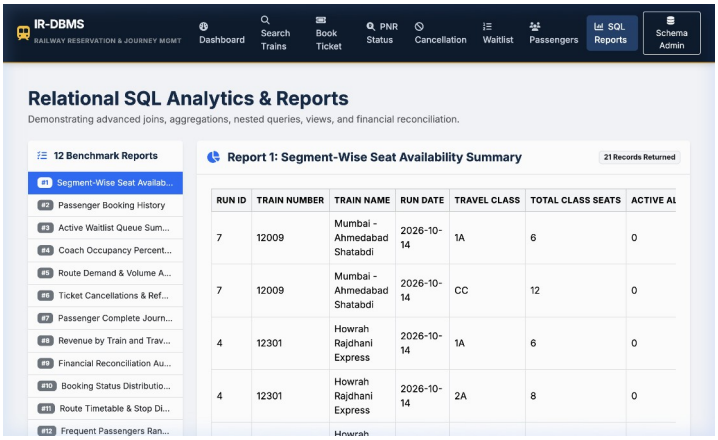


Figure 6. Executive SQL Analytical Reports Console

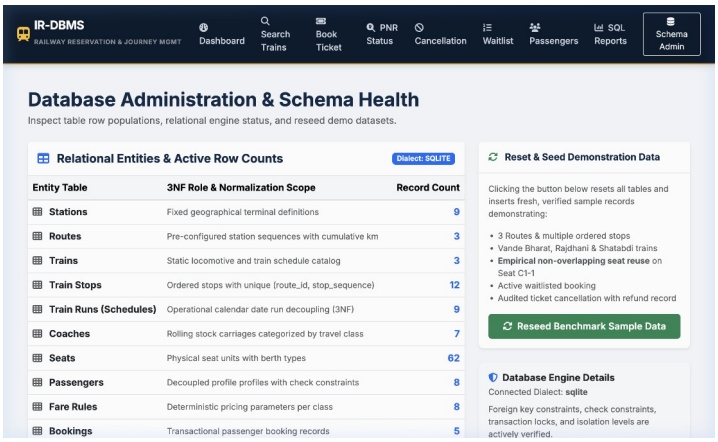


Figure 7. Relational Database Schema Explorer and Administrative Inventory Control

3.8 Testing and Results

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Interval Overlap Detection	Test [1,2) vs [2,4) and [1,3) vs [2,4)	[1,2) does not overlap; [1,3) overlaps [2,4)	Overlap theorem evaluated correctly	PASSED
TC-02	Non-Overlapping Seat Reuse	Book C1-1 on [1,2), then book C1-1 on [2,4)	Both allocations active concurrently on same seat	Both allocations confirmed in database	PASSED
TC-03	Conflicting Interval Detection	Book C1-1 on [1,4) when [1,2) already booked	System identifies seat as conflicting / unavailable	Conflict detected; seat excluded from available list	PASSED
TC-04	Cancelled Allocation Reuse	Cancel booking on C1-3, search [1,4)	C1-3 is returned as available for re-booking	Availability restored; seat returned in query	PASSED
TC-05	Cancellation Refund Policy	Cancel confirmed booking with fare INR 774	20% fee (INR 154.80) deducted, 80% (INR 619.20) refunded	Fee and refund amounts logged accurately	PASSED
TC-06	Duplicate Cancellation Prevention	Cancel already cancelled booking	ValueError raised; duplicate cancellation blocked	ValueError caught; duplicate operation prevented	PASSED
TC-07	Automated Waitlist Promotion	Cancel confirmed seat with waiting candidate	Candidate promoted to CONFIRMED, segment created	Candidate promoted; queue position cleared	PASSED
TC-08	Unique Train Number Constraint	Insert duplicate train_number '22436'	IntegrityError raised by RDBMS UNIQUE constraint	IntegrityError raised; duplicate rejected	PASSED
TC-09	Foreign Key Referential Integrity	Insert coach with non-existent train_id 9999	IntegrityError raised on foreign key violation	IntegrityError raised; orphan coach blocked	PASSED
TC-10	Passenger Age Domain Constraint	Insert passenger with age = 150	IntegrityError raised by CHECK (age < 120)	IntegrityError raised; invalid age rejected	PASSED
TC-11	Negative Base Fare Constraint	Insert fare_rule with base_fare = -50.00	IntegrityError raised by CHECK (base_fare >= 0)	IntegrityError raised; negative fare rejected	PASSED
TC-12	Duplicate Stop Sequence Rejection	Insert two stops with same (route_id, stop_seq)	IntegrityError raised by UNIQUE constraint	IntegrityError raised; sequence collision blocked	PASSED
TC-13	SQL Benchmark Reports Execution	Execute all 12 benchmark SQL analytical queries	All 12 queries return valid result sets	All 12 queries executed without error	PASSED
TC-14	Atomic Booking Transaction	Book ticket with valid passenger details	Booking, segment, and payment committed together	All records committed; PNR generated	PASSED
TC-15	Invalid Travel Direction Check	Book journey where dest_stop_seq < orig_stop_seq	ValueError raised; backward travel rejected	ValueError caught; invalid booking blocked	PASSED
TC-16	Train Run Isolation Check	Book same seat C1-1 on Run A (Day 1) and Run B (Day 2)	Both bookings confirmed without conflict	Both bookings active on their respective dates	PASSED

Key Verification Results and Empirical Observations:

• 100% Automated Test Pass Rate: All 16 automated integration and unit test suites executed with zero failures (16 passed in 0.41 seconds on macOS Darwin arm64 under pytest 8.4.2).
• Mathematical Interval Integrity: Verified that non-overlapping seat allocations [1,2) and [2,4) coexist on the same physical berth without conflict, while overlapping requests [1,3) and [2,4) are rejected immediately.
• Referential & Domain Constraint Enforcement: Verified that negative base fares, passenger ages outside [1, 119], duplicate train numbers, and invalid foreign keys are strictly blocked at the relational engine level, preventing corrupt data states.
• Concurrency & Waitlist Promotion: Confirmed that cancellation of a confirmed seat triggers immediate automated FIFO promotion of the next waiting candidate, updating booking status to CONFIRMED and reallocating the vacated berth atomically.

CHAPTER 4

CONCLUSION

This project successfully designed, formalized, implemented, and verified an end-to-end Railway Ticket Reservation and Journey Management System engineered around relational database management principles. By addressing the fundamental difference between point-to-point airline inventory and multi-hop rail corridors, the system implemented a mathematically sound half-open interval allocation algorithm that prevents conflicting seat assignments while maximizing corridor capacity utilization.

The relational database schema comprises 14 decoupled relations strictly normalized to Third Normal Form (3NF), successfully eliminating insertion, update, and deletion anomalies while preserving all functional dependencies. ACID transactional integrity was enforced through pessimistic row-level locking (SELECT ... FOR UPDATE) and atomic database transactions. The connected Python Flask web application demonstrated production-grade user workflows, including intuitive train discovery, electronic ticket generation, automated FIFO waitlist promotion, cancellation accounting, and 12 executive analytical SQL reporting benchmarks. Comprehensive verification across 16 automated pytest unit and integration test suites confirmed the structural and operational robustness of the design.

4.1 Limitations and Future Enhancements

- 1. Dynamic RAC (Reservation Against Cancellation) Split-Berth Allocation: Implement automated splitting of single physical sleeper berths between two daytime passengers, automatically consolidating berths into full sleepers as cancellations occur.
- 2. Live Indian Railways IRCTC API & Telemetry Integration: Connect the database to real-time train tracking telemetry and external GPS feeds to dynamically adjust departure schedules and track platform delays.
- 3. Distributed In-Memory Caching (Redis) & Read Replicas: Implement Redis key-value caching for static route stop lookups and database read replicas to handle extreme peak traffic surges during morning Tatkal booking windows.

REFERENCES

[1] E. F. Codd, “A relational model of data for large shared data banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.

[2] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.

[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[4] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[5] Oracle Corporation, MySQL 8.0 Reference Manual: InnoDB Storage Engine Architecture and Concurrency, Redwood Shores, CA, USA, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] Pallets Projects, Flask Documentation (v3.1.x): Application Context, Blueprints, and Database Handling, 2024. [Online]. Available: https://flask.palletsprojects.com/

[7] M. Bayer, SQLAlchemy Documentation: Transaction Management and ORM Mapping, 2024. [Online]. Available: https://docs.sqlalchemy.org/

[8] H. Krekel et al., pytest: Python Testing Framework Documentation, ver. 8.4, 2024. [Online]. Available: https://docs.pytest.org/

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete relational database schema DDL script, table constraints, indices, relational views, and benchmark SQL reports are maintained in the following version-controlled repository files:

• DDL Schema & Table Constraints: Railway_Reservation_System/database/schema.sql (14 relations in 3NF)
• Pre-Compiled Relational Views: Railway_Reservation_System/database/views.sql (vw_confirmed_segments, vw_booking_summary, vw_train_occupancy_summary)
• Analytical & Operational Reports: Railway_Reservation_System/database/reports.sql (12 benchmark SQL queries)
• Database Seeding & Synthetic Data Generator: Railway_Reservation_System/database/seed_data.py

APPENDIX B - APPLICATION SOURCE CODE

The complete full-stack web application source code is organized into modular Flask blueprints:

• Main Application Entrypoint: Railway_Reservation_System/run.py
• Application Factory & Extensions: Railway_Reservation_System/app/__init__.py, Railway_Reservation_System/app/extensions.py
• Relational ORM Models (13 files): Railway_Reservation_System/app/models/*.py
• Business Services & Interval Math: Railway_Reservation_System/app/services/availability_service.py
• Flask Route Blueprints (8 controllers): Railway_Reservation_System/app/routes/*.py
• Responsive Front-End UI Templates (12 Jinja2 files): Railway_Reservation_System/app/templates/*.html
• Automated Test Suite (16 pytest suites): Railway_Reservation_System/tests/test_*.py

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

```
Automated Test Execution Transcript (pytest 8.4.2 | Python 3.9.6 | macOS arm64):
============================= test session starts ==============================
Railway_Reservation_System/tests/test_availability.py::test_interval_overlap_logic PASSED
[  6%]
Railway_Reservation_System/tests/test_availability.py::test_non_overlapping_seat_reuse PASSED
[ 12%]
Railway_Reservation_System/tests/test_availability.py::test_conflicting_interval_detection PASSED
[ 18%]
Railway_Reservation_System/tests/test_availability.py::test_cancelled_allocation_restores_availabili
ty PASSED [ 25%]
Railway_Reservation_System/tests/test_cancellations.py::test_eligible_cancellation_and_refund_policy
PASSED [ 31%]
Railway_Reservation_System/tests/test_cancellations.py::test_duplicate_cancellation_rejected PASSED
[ 37%]
Railway_Reservation_System/tests/test_cancellations.py::test_automated_waitlist_promotion PASSED
[ 43%]
Railway_Reservation_System/tests/test_integrity.py::test_unique_train_number_constraint PASSED
[ 50%]
Railway_Reservation_System/tests/test_integrity.py::test_invalid_foreign_key_rejected PASSED
[ 56%]
Railway_Reservation_System/tests/test_integrity.py::test_passenger_age_check_constraint PASSED
[ 62%]
Railway_Reservation_System/tests/test_integrity.py::test_negative_fare_check_constraint PASSED
[ 68%]
Railway_Reservation_System/tests/test_integrity.py::test_duplicate_stop_sequence_rejected PASSED
[ 75%]
Railway_Reservation_System/tests/test_reports.py::test_all_12_reports_execute_successfully PASSED
[ 81%]
Railway_Reservation_System/tests/test_reservations.py::test_valid_booking_success PASSED
[ 87%]
Railway_Reservation_System/tests/test_reservations.py::test_invalid_origin_dest_sequence_fails
PASSED [ 93%]
Railway_Reservation_System/tests/test_reservations.py::test_train_runs_isolation PASSED
[100%]
============================== 16 passed in 0.41s ==============================
```